CSP-S 2021 (Senior)

2021-10-23, 14:30–18:30 (4 hours)

Problem 1: Jet Bridge Allocation (廊桥分配)

Problem file name: `airport`. Time limit: 1 s. Memory limit: 512 MB.

Problem description

When an airplane arrives at an airport, it can either park at a jet bridge next to the terminal, or park at a remote stand located at the edge of the airport. Passengers generally prefer to park at a jet bridge, since this saves them the trouble of taking a shuttle bus to the terminal. However, because the number of jet bridges is limited, this wish cannot always be fulfilled.

The airport is divided into a domestic zone and an international zone. Domestic flights can only park in the domestic zone, and international flights can only park in the international zone. Some of the jet bridges belong to the domestic zone, and the remaining jet bridges belong to the international zone.

City L has built a new airport with a total of n jet bridges. The airport has decided that the use of jet bridges follows the "first come, first served" principle: after each airplane arrives, if the corresponding zone (domestic / international) still has a free jet bridge, it parks at a jet bridge, otherwise it parks at a remote stand (assume that the number of remote stands is sufficient). The airport has only one runway, so no two airplanes arrive at the same time.

Now, given the arrival and departure times of the airplanes in a coming period of time, you are in charge of allocating the n jet bridges to the domestic zone and the international zone, so that the number of airplanes that park at a jet bridge is maximized.

Input

Read the input from the file `airport.in`.

The first line of the input contains 3 positive integers n, m_1, m_2, denoting the number of jet bridges, the number of domestic flights and the number of international flights, respectively.

The next m_1 lines contain the information of the domestic flights. The i-th of them contains 2 positive integers $a_{1,i}, b_{1,i}$, denoting the arrival time and the departure time of a domestic flight, respectively.

The next m_2 lines contain the information of the international flights. The i-th of them contains 2 positive integers $a_{2,i}, b_{2,i}$, denoting the arrival time and the departure time of an international flight, respectively.

The integers on each line are separated by spaces.

Output

Write the output to the file `airport.out`.

Output one positive integer, the maximum number of airplanes that can park at a jet bridge.

Sample 1 input

```
3 5 4
1 5
3 8
6 10
9 14
13 18
2 11
4 15
7 17
12 16
```

Sample 1 output

```
7
```

Explanation of sample 1

The original statement contains the following figure (Figure 1: sample figure), reproduced here as a table. In the original figure the four international-flight cells of the row "1 / 2" are shaded.

Jet bridge allocation		Domestic flights					International flights				Airplanes parked
Domestic	International	1, 5	3, 8	6, 10	9, 14	13, 18	2, 11	4, 15	7, 17	12, 16	at a jet bridge
0	3	×	×	×	×	×	✓	✓	✓	✓	4
1	2	✓	×	✓	×	✓	✓	✓	×	✓	6
2	1	✓	✓	✓	✓	✓	✓	×	×	✓	7
3	0	✓	✓	✓	✓	✓	×	×	×	×	5

In the figure, we represent an airplane by the pair of its arrival and departure times. For example, $(1, 5)$ denotes the airplane that arrives at time 1 and departs at time 5. A ✓ means that the airplane parks at a jet bridge, and a × means that the airplane parks at a remote stand.

We take the computation of the shaded part of the table as an example to explain the meaning of the table. In this part, the international zone has 2 jet bridges, and the 4 international flights arrive in the following order:

1. First, $(2, 11)$ arrives at time 2 and parks at a jet bridge.
2. Then, $(4, 15)$ arrives at time 4 and parks at the other jet bridge.
3. Next, $(7, 17)$ arrives at time 7. At this moment the first 2 airplanes have not departed yet and still occupy the jet bridges, while the international zone has only 2 jet bridges, so it can only park at a remote stand.
4. Finally, $(12, 16)$ arrives at time 12. At this moment the airplane (2 11) has already departed, so there is 1 free jet bridge, and this airplane can park at a jet bridge.

According to the results computed in the table, when the domestic zone is allocated 2 jet bridges and the international zone is allocated 1 jet bridge, the number of airplanes parked at a jet bridge is maximized, with 7 airplanes in total.

Sample 2 input

```
2 4 6
20 30
40 50
21 22
41 42
1 19
2 18
3 4
5 6
7 8
9 10
```

Sample 2 output

```
4
```

Explanation of sample 2

When the domestic zone is allocated 2 jet bridges and the international zone is allocated 0 jet bridges, the number of airplanes parked at a jet bridge is maximized, with 4 airplanes in total, i.e., all domestic flights can park at a jet bridge.

Note that in this problem the use of jet bridges follows the "first come, first served" principle. If the international zone had only 1 jet bridge, it would be occupied by the airplane $(1, 19)$, and would not be used one after another by the 4 airplanes $(3, 4)$, $(5, 6)$, $(7, 8)$, $(9, 10)$.

Sample 3

See `airport/airport3.in` and `airport/airport3.ans` in the contestant directory. On QOJ this sample is provided as the downloadable files `ex_airport3.in` and `ex_airport3.ans`.

Constraints

For 20% of the data, $1 \le n \le 100$, $1 \le m_1 + m_2 \le 100$.

For 40% of the data, $1 \le n \le 5000$, $1 \le m_1 + m_2 \le 5000$.

For 100% of the data, $1 \le n \le 100000$, $1 \le m_1 + m_2 \le 100000$.

All $a_{1,i}, b_{1,i}, a_{2,i}, b_{2,i}$ are pairwise distinct positive integers whose values do not exceed 10^8.

It is guaranteed that $\forall i \in [1, n]$, $a_{1,i} < b_{1,i}$, $a_{2,i} < b_{2,i}$.

Problem 2: Bracket Sequence (括号序列)

Problem file name: `bracket`. Time limit: 1 s. Memory limit: 512 MB.

Problem description

Little w met the following problem in a contest: a valid bracket sequence of length n has some of its positions already determined and some positions not yet determined; find how many such bracket sequences there are.

Being a veteran of many contests, little w of course solved this problem at a glance. Moreover, he felt that putting such a simple template problem in an official contest was far too childish, so he strengthened the problem and tossed it to little c.

Specifically, little w defines a "super bracket sequence" to be a string consisting of the characters `(`, `)`, `*`, and, for some given constant k, gives the following definition of a "valid super bracket sequence":

1. `()` and `(S)` are both valid super bracket sequences, where `S` denotes any non-empty string consisting only of at most k characters `*` (`S` has the same meaning in the following two rules);
2. if the strings `A` and `B` are both valid super bracket sequences, then the strings `AB` and `ASB` are both valid super bracket sequences, where `AB` denotes the string formed by concatenating the string `A` and the string `B`;
3. if the string `A` is a valid super bracket sequence, then the strings `(A)`, `(SA)`, `(AS)` are all valid super bracket sequences.
4. All valid super bracket sequences can be obtained by the above 3 rules.

For example, if $k = 3$, then the string `((**()*(*))*)(***)` is a valid super bracket sequence, but the strings `*()`, `(*()*)`, `((**))*)`, `(****(*))` are not. In particular, the empty string is not considered a valid super bracket sequence either.

Now you are given a super bracket sequence of length n, in which the characters at some positions are already determined, and the characters at the other positions are not yet determined (denoted by `?`). Little w wants to compute: how many ways are there to determine all the undetermined characters, one by one, so that the resulting string is a valid super bracket sequence?

Poor little c cannot solve this problem, so he has to ask you for help.

Input

Read the input from the file `bracket.in`.

Line 1: 2 positive integers n, k.

Line 2: a string S of length n consisting only of `(`, `)`, `*`, `?`.

Output

Write the output to the file `bracket.out`.

Output one non-negative integer, the answer modulo $10^9 + 7$.

Sample 1 input

```
7 3
(*??*??
```

Sample 1 output

```
5
```

Explanation of sample 1

The following ways are valid:

```
(**)*()
(**(*))
(*(**))
(*)**()
(*)(**)
```

Sample 2 input

```
10 2
???(*??(?)
```

Sample 2 output

```
19
```

Sample 3

See `bracket/bracket3.in` and `bracket/bracket3.ans` in the contestant directory. On QOJ this sample is provided as the downloadable files `ex_bracket3.in` and `ex_bracket3.ans`.

Sample 4

See `bracket/bracket4.in` and `bracket/bracket4.ans` in the contestant directory. On QOJ this sample is provided as the downloadable files `ex_bracket4.in` and `ex_bracket4.ans`.

Constraints

Test case number	$n \leq$	Special property
$1 \sim 3$	15	
$4 \sim 8$	40	none
$9 \sim 13$	100	
$14 \sim 15$	500	the string S contains only the character `?`
$16 \sim 20$	500	none

In the original table, the cell "none" spans the test cases $1 \sim 13$ and the cell "500" spans the test cases $14 \sim 20$.

For 100% of the data, $1 \leq k \leq n \leq 500$.

Problem 3: Palindrome (回文)

Problem file name: `palin`. Time limit: 1 s. Memory limit: 512 MB.

Problem description

You are given a positive integer n and an integer sequence $a_1, a_2, \ldots, a_{2n}$, in which each of $1, 2, \ldots, n$ appears exactly 2 times among these $2n$ numbers. Now $2n$ operations are performed, with the goal of creating a sequence $b_1, b_2, \ldots, b_{2n}$ also of length $2n$. Initially b is the empty sequence, and each time one of the following two operations may be performed:

1. append the first element of the sequence a to the end of b, and remove it from a;
2. append the last element of the sequence a to the end of b, and remove it from a.

Our goal is to make b a palindromic sequence, i.e., to make it satisfy $b_i = b_{2n+1-i}$ for all $1 \leq i \leq n$. Please determine whether this goal can be achieved. If it can, output the lexicographically smallest operation scheme, as specified in the section "Output".

Input

Read the input from the file `palin.in`.

Each test case contains multiple test data sets.

The first line of the input contains an integer T, the number of test data sets.

The first line of each test data set contains a positive integer n, and the second line contains $2n$ space-separated integers $a_1, a_2, \ldots, a_{2n}$.

Output

Write the output to the file `palin.out`.

For each test data set output one line with the answer.

If a palindromic sequence cannot be produced, output one line containing `-1`. Otherwise output one line containing a string of length $2n$ consisting of the characters `L` or `R` (without spaces), where `L` denotes operation 1 (removing the first element) and `R` denotes operation 2. You need to output the lexicographically smallest one among the strings corresponding to all schemes.

The rule for comparing lexicographic order is as follows: a string $s_{1..2n}$ is lexicographically smaller than a string $t_{1..2n}$, both of length $2n$, if and only if there exists an index $1 \leq k \leq 2n$ such that $s_i = t_i$ for all $1 \leq i < k$ and $s_k < t_k$.

Sample 1 input

```
2
5
4 1 2 4 5 3 1 2 3 5
3
3 2 1 2 1 3
```

Sample 1 output

```
LRRLLRRRRL
-1
```

Explanation of sample 1

In the first data set, the produced sequence b is `4 5 3 1 2 2 1 3 5 4`, which is seen to be a palindromic sequence.

Another possible operation scheme is `LRRLLRRRRR`, but it is lexicographically larger than the answer scheme.

Sample 2

See `palin/palin2.in` and `palin/palin2.ans` in the contestant directory. On QOJ this sample is provided as the downloadable files `ex_palin2.in` and `ex_palin2.ans`.

Constraints

Let $\sum n$ denote the sum of n over all T test data sets.

For all test cases it is guaranteed that $1 \le T \le 100$, $1 \le n$, $\sum n \le 5 \times 10^5$.

Test case number	T	n	$\sum n$	Special property
$1 \sim 7$	≤ 10	≤ 10	≤ 50	
$8 \sim 10$	≤ 100	≤ 20	≤ 1000	
$11 \sim 12$	≤ 100	≤ 100	≤ 1000	no
$13 \sim 15$	≤ 100	≤ 1000	≤ 25000	
$16 \sim 17$	$= 1$	$\le 5 \times 10^5$	$\le 5 \times 10^5$	
$18 \sim 20$	≤ 100	$\le 5 \times 10^5$	$\le 5 \times 10^5$	yes
$21 \sim 25$	≤ 100	$\le 5 \times 10^5$	$\le 5 \times 10^5$	no

In the original table, the cell "≤ 100" in the column T spans the test cases $8 \sim 15$ and again $18 \sim 25$, the cell "≤ 1000" in the column $\sum n$ spans the test cases $8 \sim 12$, the cells "$\le 5 \times 10^5$" span the test cases $16 \sim 25$, and the cell "no" in the last column spans the test cases $1 \sim 17$.

Special property: if we repeatedly delete two adjacent equal numbers from a, there is a way to delete the whole sequence (for example $a = [1, 2, 2, 1]$).

Problem 4: Traffic Planning (交通规划)

Problem file name: `traffic`. Time limit: 3 s. Memory limit: 512 MB.

Problem description

You are given n horizontal lines and m vertical lines in the plane. They intersect to form a grid with n rows and m columns. The intersection point of the r-th horizontal line from the top and the c-th vertical line from the left is called the grid point (r, c). The segment between any two horizontally or vertically adjacent grid points of the grid is called an edge, and each edge has a non-negative integer edge weight.

T queries are performed, each of the following form:

k additional points are given (the k of the T queries may differ), each of which lies on a ray that starts at the boundary of the grid and goes outwards. All rays that start at the boundary of the grid and go outwards are numbered 1 to $2n + 2m$ in the order top-left – top-right – bottom-right – bottom-left – top-left, as in the figure below.

The original statement shows here a figure (Figure 2: numbering of the rays), which is described in words as follows. The rays going upwards from the top boundary are numbered 1 to m from left to right (the ray at column c has number c). The rays going rightwards from the right boundary are numbered $m+1$ to $m+n$ from top to bottom (the ray at row r has number $m+r$). The rays going downwards from the bottom boundary are numbered $m+n+1$ to $2m+n$ from right to left (the ray at column c has number $2m+n+1-c$). The rays going leftwards from the left boundary are numbered $2m+n+1$ to $2m+2n$ from bottom to top (the ray at row r has number $2m+2n+1-r$).

For each query, the rays on which the different additional points lie are pairwise distinct. The segment between each additional point and its nearest grid point is also called an edge, and also has a non-negative integer edge weight (note that a grid point at a corner may be connected to two additional points at the same time).

Given the color (black or white) of each additional point, please color every grid point inside the grid with one of the two colors black and white, so that the sum of the edge weights of all edges whose two endpoints have different colors is minimized. Please output this minimum sum of edge weights.

Input

Read the input from the file `traffic.in`.

The first line contains 3 positive integers n, m, T, denoting the numbers of horizontal and vertical lines, and the number of queries, respectively.

The next $n-1$ lines each contain m non-negative integers. The j-th non-negative integer $x1_{i,j}$ on the i-th of these lines denotes the edge weight between (i,j) and $(i+1,j)$.

The next n lines each contain $m-1$ non-negative integers. The j-th non-negative integer $x2_{i,j}$ on the i-th of these lines denotes the edge weight between (i,j) and $(i,j+1)$.

Next the T queries are given in order. The i-th query begins with a line containing a positive integer k_i, the total number of additional points in this query. The next k_i lines each contain three non-negative integers. The j-th of these lines contains $x3_{i,j}, p_{i,j}, t_{i,j}$ in this order, denoting the edge weight between the additional point and its adjacent grid point, the number of the ray it lies on, and the color of the additional point (0 for white, 1 for black). It is guaranteed that within one query the $p_{i,j}$ are pairwise distinct.

The integers on each line are separated by spaces.

Output

Write the output to the file `traffic.out`.

Output T lines. The i-th line contains one non-negative integer, the minimum sum of edge weights of edges whose two endpoints have different colors after the coloring for the i-th query.

Sample 1 input

```
2 3 1
9 4 7
3 8
10 5
2
19 3 1
17 9 0
```

Sample 1 output

```
12
```

Explanation of sample 1

Optimal scheme: $(1,3),(1,2),(2,3)$ are black; $(1,1),(2,1),(2,2)$ are white.

Sample 2

See `traffic/traffic2.in` and `traffic/traffic2.ans` in the contestant directory. On QOJ this sample is provided as the downloadable files `ex_traffic2.in` and `ex_traffic2.ans`.

Sample 3

See `traffic/traffic3.in` and `traffic/traffic3.ans` in the contestant directory. On QOJ this sample is provided as the downloadable files `ex_traffic3.in` and `ex_traffic3.ans`.

Sample 4

See `traffic/traffic4.in` and `traffic/traffic4.ans` in the contestant directory. On QOJ this sample is provided as the downloadable files `ex_traffic4.in` and `ex_traffic4.ans`.

Sample 5

See `traffic/traffic5.in` and `traffic/traffic5.ans` in the contestant directory. On QOJ this sample is provided as the downloadable files `ex_traffic5.in` and `ex_traffic5.ans`.

Constraints

Test case number	$n, m \leq$	$k_i \leq$
$1, 2$	5	50
$3, 4, 5$	18	2
$6, 7, 8$	18	50
$9, 10$	10^2	2
$11, 12$	10^2	50
$13, 14, 15, 16$	500	2
$17, 18, 19, 20$	500	50

In the original table, each value of the column $n, m \leq$ is a single cell spanning the two rows of its group.

For all data, $2 \leq n, m \leq 500$, $1 \leq T \leq 50$, $1 \leq k_i \leq \min\{2(n+m), 50\}$, $1 \leq \sum_{i=1}^{T} k_i \leq 50$, $0 \leq x \leq 10^6$, $1 \leq p \leq 2(n+m)$, $t \in \{0, 1\}$.

It is guaranteed that for every $i \in [1, T]$, the $p_{i,j}$ are pairwise distinct.

Source

CSP-S 2021 (CCF CSP-J/S 2021 second round, senior group), held on 2021-10-23. The statements were taken from the official statement PDF embedded in the following QOJ problem pages (contest "CSP-S 2021" on QOJ):

- Problem 1: `https://qoj.ac/problem/1768`
- Problem 2: `https://qoj.ac/problem/1769`
- Problem 3: `https://qoj.ac/problem/1770`
- Problem 4: `https://qoj.ac/problem/1771`